近似演算法詳解

35-E01 Matching 下界與加權反例

推導

先選中心c與某葉a，所有邊皆被c覆蓋。未加權ALG=2、OPT=1，比值2，與matching邊數1的下界相符。加權時輸出成本1+100=101，最優只取c成本1，比例101。

Matching的每條邊只保證cover須選至少一端，並不保證演算法所選兩端的權重≤最優所選端的兩倍。權重可任意懸殊，將葉權改為W就得W+1比值，無固定常數上界。

結論與檢核

「兩個端點／至少一個端點」控制的是個數，不能直接控制加權和。加權近似需另建權重下界及相應選擇規則，例如LP捨入。

35-E02 Metric 的 Walk 與 Tour

推導

MST鏈邊AD=2、DC=3、CB=2，總7。全圖六條邊成本為2, 2, 3, 3, 5, 5，任何生成樹有3條邊，因此成本至少2+2+3=7，此鏈達下界。

完整walk A,D,C,B,C,D,A成本2+3+2+2+3+2=14。首次出現tour A,D,C,B,A成本2+3+2+3=10。封閉tour要走到x最小0與最大3，x方向總變動至少6；y方向範圍0到2，總變動至少4，故成本至少10，這個tour即最優。

結論與檢核

Walk14、tour10、OPT10為三個不同量。聲稱OPT=6已違反MST≤OPT的7下界，也違反由座標推出的10下界；不能先接受不一致數據再套2倍比率。

35-E03 加權 Set-cover 與收費

推導

初始每單位成本S1=2/3、S2=2/3、S3=1/4，取S3，元素1, 2, 4, 5各付1/4，剩3, 6。第二輪S1、S2各新增1、比值皆2，取S1，元素3付2，剩6。第三輪取S2，元素6付2。ALG=5。

S1與S2兩組成本4並覆蓋全集；任何用S3的cover仍須S1補3、S2補6，成本5。沒有單組可行，因此OPT=4。每個最佳集合上的charge：S1為1/4+1/4+2=5/2，S2同樣5/2；皆≤w(S)H3=2・11/6=11/3。

結論與檢核

實際比率5/4≤H4=25/12（全family最大組大小d=4）。計費合計5與實際成本一致；在最佳集合上加總得到5≤(H3)・4≤(H4)・4，未把未覆蓋元素遺漏。

近似演算法詳解 （續）

35-E04 期望目標值不是期望比值

推導

c=1時四clause皆真，Y=4；c=0時四個(a,b)符號組合中，恰有一clause為假，Y=3。兩種各機率1/2，所以E[Y]=7/2、OPT=4，但E[OPT/Y]=(1/2)(4/4)+(1/2)(4/3)=7/6>8/7。這直接反證把8/7期望目標值保證誤寫成期望比值保證。

條件期望順序a,b,c：a=0或1皆為7/2，平手取0；固定a=0後，b=0或1也皆7/2，取0。此時c=0只有3、c=1有4，選1，最後assignment(0, 0, 1)滿足4clause。

結論與檢核

四事件共享c，顯然不是獨立；E[ΣI_j]=ΣE[I_j]本來不需獨立。需要獨立的是用同一clause三個不同變數的真假算其全假機率1/8。

35-E05 LP 下界、整數最優與捨入

推導

對ab、ac、bc三條限制分別乘1, 2, 3後相加，左邊為3x_a+4x_b+5x_c，右邊6。因此任一LP可行解成本≥6，半值解成本(3+4+5)/2=6達下界，LP^*=6。

整數cover至少取兩點，三種成本7, 8, 9，所以OPT=7。Threshold≥1/2使三點全取，ALG=12=2LP^*≤2OPT=14，實際比率12/7。

結論與檢核

LP最優6不同於整數最優7。邊乘數非負且相加後每個變數係數恰好是權重，因此這個下界證明不需要猜測「半值看起來對稱」。

35-E06 FPTAS 的四輪整數 Trace

推導

n=4，δ=(2/5)/8=1/20。

i	merge_unique	TRIM	刪超過305後
1	0, 102	0, 102	0, 102
2	0, 98, 102, 200	0, 98, 200	0, 98, 200
3	0, 98, 195, 200, 293, 395	0, 98, 195, 293, 395	0, 98, 195, 293
4	0, 98, 101, 195, 199, 293, 296, 394	0, 98, 195, 293, 394	0, 98, 195, 293

近似演算法詳解 （續）

例如102≤98・1.05=102.9，200≤195・1.05=204.75，296≤293・1.05=307.65，因此各被較小代表刪除。輸出293=98+195，來自索引2, 3。完整精確可行和集合為{0, 98, 101, 102, 195, 199, 200, 203, 293, 296, 297, 301}，故OPT=301=102+98+101。

結論與檢核

293≤305且301/293≈1.0273≤1.4，亦293≥0.6・301。清單每個代表仍是真實子集合和；ε是全局參數，只有δ=1/20用於每輪。

35-E07 代表方向與負數的反例

推導

處理4, 5後精確可行和為0, 4, 5, 9。若只把4刪掉並用較大5代替，清單0, 5, 9；再加入6，候選0, 5, 6, 9, 11, 15，刪>10後最大9。真正可行最優4+6=10已被丟失。歸納要求代表z≤y以保證z+未來項仍不超額；改成較大代表就無此保證。

若輸入8,−3且t=5，第一輪刪去8，只剩0；下一輪只能得到0或−3，卻漏了8+(−3)=5。正數時「超額之後永遠超額」成立，負數時不成立。

結論與檢核

這些反例否定的是原操作／原證明的直接擴張；並未證明所有允許負數或不同代表設計的演算法都不可能近似。若改模型，需重新定義可行性、狀態與誤差分析。

35-E08 Gap 歸約與加常數的陷阱

推導

n=5、ρ=2，取M=2n+1=11。有HC時OPT=5，假設的2近似必回≤10；無HC時任何tour至少一條權11、另4條至少1，故成本≥15>10。以10作門檻即可區分，且數值11只有4個binary bits，沒有超長輸出。

對一般非負權c，令K=max c，c'(u,v)=c(u,v)+K。因c(u,w)≤K、c(u,v)+c(v,w)≥0，有c(u,w)+K≤2K≤c(u,v)+c(v,w)+2K，故c'滿足三角不等式。每個n點tour都恰加nK，所以最佳tour集合不變。

然而c'(H)≤ρOPT'只推出c(H)+nK≤ρ(OPT+nK)，即c(H)≤ρOPT+(ρ−1)nK。額外項可比OPT大很多，無法推出原ρ近似。

結論與檢核

保持最佳解的歸約不一定保持近似比。此差別使「加常數轉metric」與一般TSP無常數近似的gap論證完全相容。

近似演算法詳解 （續）

原創驗算程式附錄: approximation_lab.py

```
"""Exact arithmetic models for approximation ratios, charges and TRIM witnesses."""
from fractions import Fraction
from dataclasses import dataclass

def maximal_cover(vertices,edges):
    chosen=set();matching=[]
    for u,v in edges:
        if u not in chosen and v not in chosen:
            chosen.update((u,v));matching.append((u,v))
    return chosen,matching

def greedy_cover(universe,sets,weights):
    remaining=set(universe);picked=[];charges={};trace=[]
    while remaining:
        candidates=[(Fraction(weights[i],len(s&remaining)),i) for i,s in enumerate(sets) if
s&remaining]
        if not candidates:raise ValueError('family does not cover the universe')
        price,i=min(candidates);new=sets[i]&remaining
        for e in new:charges[e]=price
        remaining-=new;picked.append(i);trace.append((i,sorted(new),sorted(remaining),price))
    return picked,charges,trace

@dataclass(frozen=True)
class Witness:
    value:int
    parent:object=None
    index:int=-1

def merge_nodes(a,b):
    i=j=0;out=[]
    while i<len(a) or j<len(b):
        if j==len(b) or (i<len(a) and a[i].value<=b[j].value):v=a[i];i+=1
        else:v=b[j];j+=1
        if not out or out[-1].value!=v.value:out.append(v)
    return out

def subset_trim(values,target,epsilon,collect_trace=False):
    if not 0<epsilon<1:raise ValueError('epsilon in(0,1) required')
    if target<0 or any(v<=0 for v in values):raise ValueError('nonnegative target and
positive values required')
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

近似演算法詳解 （續）

```
    if not values or target==0:return 0,[],[]
    delta=Fraction(epsilon)/(2*len(values));current=[Witness(0)];trace=[]
    for i,value in enumerate(values):
        added=[Witness(p.value+value,p,i) for p in current] if value<=target else []
        merged=merge_nodes(current,added);trimmed=[]
        for p in merged:
            if not trimmed or p.value>(1+delta)*trimmed[-1].value:trimmed.append(p)
        current=[p for p in trimmed if p.value<=target]
        if collect_trace:trace.append((i+1,[p.value for p in merged],[p.value for p in
trimmed],[p.value for p in current]))
    best=current[-1];value=best.value;indices=[]
    while best.parent is not None:indices.append(best.index);best=best.parent
    return value,indices[::-1],trace

def exact_subset(values,target):
    sums={0}
    for x in values:sums|={y+x for y in sums if y+x<=target}
    return sorted(sums)

def conditional_max3(clauses,variables):
    def expectation(assignment):
        total=Fraction(0)
        for clause in clauses:
            if any(abs(l) in assignment and assignment[abs(l)]==(l>0) for l in
clause):total+=1
            else:
                r=sum(abs(l) not in assignment for l in clause)
                total+=1-Fraction(1,2**r)
        return total
    assignment={};trace=[]
    for v in variables:
        left=expectation(assignment|{v:False});right=expectation(assignment|{v:True})
        assignment[v]=right>left;trace.append((v,left,right,assignment[v]))
    return assignment,expectation(assignment),trace

if __name__=='__main__':
    print('set cover',greedy_cover(set(range(1,7)),[{1,2,3},{4,5,6},{1,2,4,5}],[1,1,1]))
    print('subset',subset_trim([112,108,205,103],325,Fraction(2,5),True))
```

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study